

Understanding Flow Matching from a mathematical view

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

October 29, 2025

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models
- 5 Practical Implementation
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions
- 9 References

Mathematical motivation for Flow Matching

The fundamental challenge: Learn a velocity field $\mathbf{v}(\mathbf{x}, t)$ that transforms a simple base distribution $p_1(\mathbf{x})$ into the data distribution $p_0(\mathbf{x})$ through the continuity equation:

$$\partial_t p(\mathbf{x}, t) + \nabla \cdot (p(\mathbf{x}, t) \mathbf{v}(\mathbf{x}, t)) = 0. \quad (1)$$

Key advantages over score-based methods:

- **No score estimation:** Avoids computing $\nabla_{\mathbf{x}} \log p(\mathbf{x}, t)$ during training
- **Analytic supervision:** Provides closed-form velocity targets via conditional paths
- **Deterministic sampling:** Enables ODE integration with large solver steps
- **Mathematical elegance:** Direct connection to optimal transport theory
- **Computational efficiency:** Simpler training objective and faster inference

Core insight: Instead of learning to reverse a stochastic process, Flow Matching learns a deterministic flow that directly transports probability mass.

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models
- 5 Practical Implementation
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions
- 9 References

Theorem (Fokker–Planck / Forward Kolmogorov)

Let $(X_t)_{t \geq 0} \in \mathbb{R}^N$ solve the Itô SDE

$$dX_t = \mu(X_t, t) dt + \sigma(X_t, t) dW_t,$$

where $\mu : \mathbb{R}^N \times [0, \infty) \rightarrow \mathbb{R}^N$ is the drift, $\sigma : \mathbb{R}^N \times [0, \infty) \rightarrow \mathbb{R}^{N \times M}$ the diffusion, and W_t an M -dimensional standard Wiener process. Define the diffusion tensor

$$D(x, t) := \frac{1}{2} \sigma(x, t) \sigma(x, t)^\top \in \mathbb{R}^{N \times N}.$$

Then the (smooth) density $p(x, t)$ satisfies the Fokker–Planck equation

$$\partial_t p(x, t) = - \sum_{i=1}^N \frac{\partial}{\partial x_i} [\mu_i(x, t) p(x, t)] + \sum_{i,j=1}^N \frac{\partial^2}{\partial x_i \partial x_j} [D_{ij}(x, t) p(x, t)]$$

Isotropic diffusion: $\sigma(x, t) = \sigma(t) \mathbf{I}_N$

When $\sigma(x, t) = \sigma(t) \mathbf{I}_N$, the diffusion tensor becomes

$$D(x, t) = \frac{1}{2} \sigma(t)^2 \mathbf{I}_N, \quad D_{ij}(x, t) = \frac{1}{2} \sigma(t)^2 \delta_{ij}.$$

Hence the Fokker–Planck equation simplifies to the drift–diffusion form

$$\partial_t p(x, t) = -\nabla \cdot (\mu(x, t) p(x, t)) + \frac{\sigma(t)^2}{2} \Delta p(x, t)$$

where $\nabla \cdot (\mu p) = \sum_{i=1}^N \partial_{x_i} (\mu_i p)$ and $\Delta = \sum_{i=1}^N \partial_{x_i}^2$ is the Laplacian.

Conditional probability paths: mathematical framework

Definition: Given a data distribution $p_{\text{data}}(\mathbf{x}_0)$ and base distribution $p_{\text{base}}(\mathbf{x})$, we define conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0)$ such that:

$$p_t(\mathbf{x}) = \int p_{\text{data}}(\mathbf{x}_0) q_t(\mathbf{x} \mid \mathbf{x}_0) d\mathbf{x}_0, \quad t \in [0, 1], \quad (2)$$

with boundary conditions $p_0 = p_{\text{data}}$ and $p_1 = p_{\text{base}}$.

Key insight: The conditional velocity field

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) := \mathbb{E}\left[\frac{d}{dt} \mathbf{x}_t \mid \mathbf{x}_t = \mathbf{x}, \mathbf{x}_0\right] \quad (3)$$

provides *analytic supervision* for learning the marginal velocity field $\mathbf{v}(\mathbf{x}, t)$.

Mathematical advantage: No need to estimate $\nabla_{\mathbf{x}} \log p(\mathbf{x}, t)$ during training!

Connection to optimal transport: When paths are chosen optimally, Flow Matching recovers Wasserstein-2 geodesics.

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets**
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models
- 5 Practical Implementation
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions
- 9 References

Gaussian conditional paths: mathematical derivation

Assumption: Gaussian conditional paths

$$q_t(\mathbf{x} \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_t(\mathbf{x}_0), \sigma_t^2 \mathbf{I}), \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \boldsymbol{\epsilon}. \quad (4)$$

Mathematical derivation: Taking the time derivative of the path:

$$\frac{d}{dt} \mathbf{x}_t = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} = \frac{\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t}. \quad (5)$$

Conditional expectation: Since $\boldsymbol{\epsilon}$ is independent of $\mathbf{x}_t$ given $\mathbf{x}_0$, we have:

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \mathbb{E}\left[\frac{d}{dt} \mathbf{x}_t \mid \mathbf{x}_t = \mathbf{x}, \mathbf{x}_0\right] \quad (6)$$

$$= \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t \mathbb{E}[\boldsymbol{\epsilon} \mid \mathbf{x}_t = \mathbf{x}, \mathbf{x}_0] \quad (7)$$

$$= \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t \frac{\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t} \quad (8)$$

Final result:

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)).$$

Mathematical analysis of path choices

1. Linear bridge (optimal transport):

- $\mu_t(\mathbf{x}_0) = (1 - t) \mathbf{x}_0, \sigma_t = t \sigma_{\max}$
- **Mathematical property:** Minimizes Wasserstein-2 distance
- **Velocity:** $\mathbf{u}_t(\mathbf{x} | \mathbf{x}_0) = \frac{\sigma_{\max}}{t}(\mathbf{x} - \mathbf{x}_0)$
- **Advantage:** Direct connection to optimal transport theory

2. VP-like (DDPM-inspired):

- $\mu_t(\mathbf{x}_0) = \alpha(t) \mathbf{x}_0, \sigma_t = \sqrt{1 - \alpha^2(t)}$
- **Mathematical property:** Preserves energy structure
- **Velocity:** $\mathbf{u}_t(\mathbf{x} | \mathbf{x}_0) = \dot{\alpha}(t) \mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \alpha(t) \mathbf{x}_0)$
- **Advantage:** Compatible with existing diffusion model schedules

3. VE-like (SMLD-inspired):

- $\mu_t(\mathbf{x}_0) = \mathbf{x}_0, \sigma_t = \sigma(t)$ increasing
- **Mathematical property:** Preserves data structure
- **Velocity:** $\mathbf{u}_t(\mathbf{x} | \mathbf{x}_0) = \frac{\dot{\sigma}(t)}{\sigma(t)}(\mathbf{x} - \mathbf{x}_0)$
- **Advantage:** Simple form, good for high-dimensional data

Key insight: All choices provide *closed-form* $\mathbf{u}_t$ via the boxed formula!

Derivation: VE/VP target forms

VE-like. $\mu_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$. Then

$$\mathbf{u}_t = 0 + \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \mathbf{x}_0). \quad (10)$$

VP-like. $\mu_t(\mathbf{x}_0) = \alpha(t) \mathbf{x}_0$, $\sigma_t = \sqrt{1 - \alpha^2(t)}$. Using $\mathbf{u}_t = \dot{\mu}_t + (\dot{\sigma}_t/\sigma_t)(\mathbf{x} - \mu_t)$, we get

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \dot{\alpha} \mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t} \mathbf{x} - \frac{\dot{\sigma}_t}{\sigma_t} \alpha \mathbf{x}_0 \quad (11)$$

$$= \frac{\dot{\sigma}_t}{\sigma_t} \mathbf{x} + \left(\dot{\alpha} - \alpha \frac{\dot{\sigma}_t}{\sigma_t} \right) \mathbf{x}_0. \quad (12)$$

No explicit form for $\dot{\sigma}_t/\sigma_t$ is required to train; schedules determine it numerically.

Mathematical insight: These derivations show how different path choices lead to different velocity field structures, all computable analytically.

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference**
- 4 Connection to Diffusion Models
- 5 Practical Implementation
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions
- 9 References

Flow Matching: mathematical training objective

Theoretical foundation: Given conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0)$, the optimal velocity field satisfies:

$$\mathbf{v}^*(\mathbf{x}, t) = \mathbb{E}_{\mathbf{x}_0 \mid \mathbf{x}_t = \mathbf{x}} [\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0)] \quad (13)$$

Training procedure:

- 1 Sample $t \sim \mathcal{U}[0, 1]$, $\mathbf{x}_0 \sim p_{\text{data}}$, $\mathbf{x}_t \sim q_t(\cdot \mid \mathbf{x}_0)$
- 2 Compute analytic target $\mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)$ from chosen path
- 3 Minimize regression loss

Mathematical objective:

$$J_{\text{FM}} = \mathbb{E} [\| \mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0) \|^2]. \quad (14)$$

Key mathematical advantage: No score estimation needed; all targets are analytic!

Computational benefits:

- Stable gradients (no exploding/vanishing)
- Parallelizable across time steps
- No special architectural requirements

Mathematical foundation of inference

Theoretical basis: The learned velocity field $\mathbf{v}_\theta(\mathbf{x}, t)$ defines a deterministic ODE:

$$\frac{d}{dt} \mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t), \quad t \in [0, 1] \quad (15)$$

Mathematical properties:

- **Existence:** Under Lipschitz conditions, solutions exist and are unique
- **Consistency:** The ODE preserves the continuity equation
- **Stability:** Deterministic integration avoids accumulation of stochastic errors

Generation process:

- 1 Initialize $\mathbf{x}(1) \sim p_1$ (e.g., $\mathcal{N}(0, \mathbf{I})$)
- 2 Integrate ODE backwards: $\frac{d}{dt} \mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t)$, $t : 1 \rightarrow 0$
- 3 Use standard ODE solvers (Euler, RK4, Dormand-Prince, etc.)

Mathematical advantages:

- Deterministic sampling (reproducible)
- Fast with large solver steps
- No stochastic correctors needed

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models**
- 5 Practical Implementation
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions
- 9 References

Mathematical connection to diffusion models

Theoretical link: For a forward SDE $d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) dB_t$, the probability flow ODE is:

$$\frac{d}{dt} \mathbf{x} = f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}, t). \quad (16)$$

Key mathematical insight: When Flow Matching uses Gaussian paths that match the SDE's marginal distributions p_t , the learned velocity field $\mathbf{v}_\theta$ approximates the probability flow drift:

$$\mathbf{v}_\theta(\mathbf{x}, t) \approx f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}, t) \quad (17)$$

Crucial advantage: Flow Matching achieves this approximation *without* ever computing the score $\nabla_{\mathbf{x}} \log p(\mathbf{x}, t)$ during training!

Practical implications: This connection allows Flow Matching to inherit the theoretical guarantees of diffusion models while avoiding their computational bottlenecks.

Mathematical derivation: probability flow ODE

Setup: Forward SDE $d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) dB_t$ with density $p(\mathbf{x}, t)$ solving the Fokker-Planck equation:

$$\partial_t p = -\nabla \cdot (fp) + \frac{1}{2}g^2 \Delta p. \quad (18)$$

Key insight: A deterministic ODE $\dot{\mathbf{x}} = \mathbf{v}(\mathbf{x}, t)$ transports densities via the continuity equation:

$$\partial_t p + \nabla \cdot (p\mathbf{v}) = 0. \quad (19)$$

Mathematical derivation: Equating the two equations:

$$-\nabla \cdot (p\mathbf{v}) = -\nabla \cdot (fp) + \frac{1}{2}g^2 \Delta p \quad (20)$$

$$\Rightarrow \mathbf{v} = f - \frac{1}{2}g^2 \frac{\nabla p}{p} = f - \frac{1}{2}g^2 \nabla \log p. \quad (21)$$

Result: The probability flow ODE drift is $\mathbf{v} = f - \frac{1}{2}g^2 \nabla \log p$.

Connection to Flow Matching: When Flow Matching learns $\mathbf{v}_\theta$, it approximates this probability flow drift without computing the score $\nabla \log p$.

Specializing VE/VP-style paths

- VE-style: $\mu_t(\mathbf{x}_0) = \mathbf{x}_0$. Target becomes $\mathbf{u}_t = \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \mathbf{x}_0)$.
- VP-style: $\mu_t = \alpha(t)\mathbf{x}_0$, $\sigma_t = \sqrt{1 - \alpha^2(t)}$. Target is $\mathbf{u}_t = \dot{\alpha} \mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \alpha\mathbf{x}_0)$.

Averaging over $\mathbf{x}_0$ under the path recovers the probability flow drift forms used by diffusion models.

Mathematical insight: This shows that Flow Matching can recover the same velocity fields as diffusion models, but through a more direct training procedure.

Practical advantage: Flow Matching avoids the computational overhead of score estimation while achieving equivalent results.

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models
- 5 Practical Implementation**
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions
- 9 References

Complete training recipe

Step-by-step procedure:

- 1 **Sample data:** $\mathbf{x}_0 \sim p_{\text{data}}, t \sim \mathcal{U}[0, 1], \epsilon \sim \mathcal{N}(0, \mathbf{I})$
- 2 **Forward pass:** $\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \epsilon$
- 3 **Compute target:** $\mathbf{u}_t(\mathbf{x}_t | \mathbf{x}_0) = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + (\dot{\sigma}_t / \sigma_t)(\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0))$
- 4 **Regression loss:** $\|\mathbf{v}_{\theta}(\mathbf{x}_t, t) - \mathbf{u}_t(\mathbf{x}_t | \mathbf{x}_0)\|^2$

Implementation notes:

- Use standard optimizers (Adam, AdamW)
- Typical learning rates: 10^{-4} to 10^{-3}
- Training time similar to diffusion models
- No special architectural requirements beyond standard neural networks

Computational advantages:

- No score estimation during training
- Stable gradients (no exploding/vanishing)
- Parallelizable across time steps

Complete inference recipe

Generation procedure:

- 1 **Initialize:** $\mathbf{x}(1) \sim p_1$ (e.g., $\mathcal{N}(0, \mathbf{I})$)
- 2 **ODE integration:** $\dot{\mathbf{x}} = \mathbf{v}_\theta(\mathbf{x}, t)$ from $t = 1$ to $t = 0$
- 3 **Optional:** Add predictor-corrector steps for higher quality

Solver recommendations:

- **Fast:** Euler method (1-10 steps)
- **Accurate:** RK4, Dormand-Prince (10-50 steps)
- **Adaptive:** Use error control for automatic step sizing

Sampling advantages:

- Deterministic generation (reproducible results)
- Fewer function evaluations than diffusion models
- No stochastic correctors needed
- Flexible step size control

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models
- 5 Practical Implementation
- 6 Advanced Topics and Extensions**
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions
- 9 References

Conditional Flow Matching

Extension to conditional generation: Instead of unconditional paths, we can define conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0, c)$ where c represents conditioning information (e.g., class labels, text prompts).

Mathematical formulation: The conditional velocity field becomes:

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0, c) := \mathbb{E}\left[\frac{d}{dt} \mathbf{x}_t \mid \mathbf{x}_t = \mathbf{x}, \mathbf{x}_0, c\right] \quad (22)$$

Training objective:

$$J_{\text{CFM}} = \mathbb{E}\left[\|\mathbf{v}_\theta(\mathbf{x}_t, t, c) - \mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0, c)\|^2\right] \quad (23)$$

Applications:

- Class-conditional generation
- Text-to-image synthesis
- Image-to-image translation
- Style transfer

Advantage: Maintains the same computational efficiency as unconditional Flow Matching while enabling controlled generation.

Rectified Flow and Optimal Transport

Rectified Flow: A special case of Flow Matching that uses straight-line paths:

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1, \quad \mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \mathbf{x}_1 - \mathbf{x}_0 \quad (24)$$

Mathematical properties:

- **Optimal transport:** Minimizes Wasserstein-2 distance
- **Simplicity:** Constant velocity field along each path
- **Efficiency:** Fastest convergence in many cases

Connection to optimal transport: Rectified Flow recovers the optimal transport map between p_0 and p_1 when the data is well-structured.

Practical advantages:

- Fewer function evaluations during sampling
- More stable training
- Better sample quality in many domains

Flow Matching in Latent Spaces

Motivation: Instead of working directly in pixel space, Flow Matching can be applied in learned latent representations.

Mathematical setup: Given an encoder $E : \mathcal{X} \rightarrow \mathcal{Z}$ and decoder $D : \mathcal{Z} \rightarrow \mathcal{X}$:

- Learn flow in latent space: $\mathbf{v}_\theta : \mathcal{Z} \times [0, 1] \rightarrow \mathcal{Z}$
- Generate latent codes: $\mathbf{z}_0 = \mathbf{z}_1 + \int_0^1 \mathbf{v}_\theta(\mathbf{z}_t, t) dt$
- Decode to images: $\mathbf{x} = D(\mathbf{z}_0)$

Advantages:

- **Computational efficiency:** Lower-dimensional space
- **Sample quality:** Better inductive biases in latent space
- **Flexibility:** Can use different architectures for encoder/decoder

Examples: Latent Diffusion Models (LDMs), Stable Diffusion

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models
- 5 Practical Implementation
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation**
- 8 Summary and Future Directions
- 9 References

MeanFlow: Motivation

Goal: Achieve single-step (1-NFE) generation while retaining high fidelity and stability.

Problem with standard Flow Matching: Multi-step ODE integration is computationally expensive and can accumulate errors.

Key insight: Learn a *ground-truth average velocity field* that summarizes an entire time interval.

Mathematical formulation: For any pair $r < t$ and point $\mathbf{z}_t$ along the flow path:

$$\bar{\mathbf{u}}(\mathbf{z}_t, r, t) := \frac{1}{t-r} \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau) d\tau. \quad (25)$$

Properties:

- $\bar{\mathbf{u}}$ is an **underlying field** induced by $\mathbf{v}$ (no networks yet)
- As $r \rightarrow t$, $\bar{\mathbf{u}}(\mathbf{z}_t, r, t) \rightarrow \mathbf{v}(\mathbf{z}_t, t)$
- Composition over intervals holds: one big step equals two smaller consecutive steps

Definition: Average Velocity

For any pair $r < t$ and point $\mathbf{z}_t$ along the flow path:

$$\bar{\mathbf{u}}(\mathbf{z}_t, r, t) := \frac{1}{t - r} \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau) d\tau. \quad (26)$$

- $\bar{\mathbf{u}}$ is an **underlying field** induced by $\mathbf{v}$ (no networks yet).
- As $r \rightarrow t$, $\bar{\mathbf{u}}(\mathbf{z}_t, r, t) \rightarrow \mathbf{v}(\mathbf{z}_t, t)$.
- Composition over intervals holds: one big step equals two smaller consecutive steps (additivity of integrals).

MeanFlow Identity (core relation)

Starting from Eq. (26):

$$(t - r) \bar{\mathbf{u}}(\mathbf{z}_t, r, t) = \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau) d\tau. \quad (27)$$

Differentiate w.r.t. t (treating r as constant) and use the fundamental theorem of calculus:

$$\bar{\mathbf{u}}(\mathbf{z}_t, r, t) = \mathbf{v}(\mathbf{z}_t, t) - (t - r) \frac{d}{dt} \bar{\mathbf{u}}(\mathbf{z}_t, r, t)$$

(28)

where the total derivative expands to

$$\frac{d}{dt} \bar{\mathbf{u}}(\mathbf{z}_t, r, t) = \underbrace{\mathbf{v}(\mathbf{z}_t, t)}_{\frac{d\mathbf{z}_t}{dt}} \cdot \partial_{\mathbf{z}} \bar{\mathbf{u}}(\mathbf{z}_t, r, t) + \partial_t \bar{\mathbf{u}}(\mathbf{z}_t, r, t). \quad (29)$$

Interpretation: Eq. (28) links instantaneous and average velocities *without* introducing networks.

Mathematical significance: This identity enables learning average velocity fields that can be used for single-step generation.

Trainable objective

Parameterize a network $\bar{\mathbf{u}}_\theta(\mathbf{z}, r, t)$. Use the MeanFlow identity to form a target (replace $\mathbf{v}$ by sample-conditional $\mathbf{v}_t$):

$$\text{Target: } \bar{\mathbf{u}}_{\text{tgt}} = \mathbf{v}_t - (t - r) \underbrace{(\mathbf{v}_t \cdot \partial_{\mathbf{z}} \bar{\mathbf{u}}_\theta)}_{\text{JVP}} + \partial_t \bar{\mathbf{u}}_\theta. \quad (30)$$

$$\mathcal{L}(\theta) = \mathbb{E} \left\| \bar{\mathbf{u}}_\theta(\mathbf{z}_t, r, t) - \text{sg}(\bar{\mathbf{u}}_{\text{tgt}}) \right\|_2^2. \quad (31)$$

Key components:

- $\text{sg}(\cdot)$: stop-gradient; avoids higher-order differentiation
- JVP (Jacobian-vector product) computed efficiently via autodiff
- If $r = t$, the correction term vanishes $\Rightarrow$ reduces to standard Flow Matching

Mathematical insight: This objective learns to predict average velocities over time intervals, enabling single-step generation.

Pseudocode (training)

MeanFlow training algorithm:

- 1 Sample $t, r \sim \text{LogitNormal}$; set $t \leftarrow \max(r, t)$, $r \leftarrow \min(r, t)$
- 2 Sample $\mathbf{x} \sim p_{\text{data}}$, $\epsilon \sim \mathcal{N}(0, I)$; set $\mathbf{z}_t = (1 - t)\mathbf{x} + t\epsilon$, $\mathbf{v}_t = \epsilon - \mathbf{x}$
- 3 Compute $(\bar{\mathbf{u}}_\theta, \frac{d}{dt}\bar{\mathbf{u}}_\theta) = \text{JVP}(\bar{\mathbf{u}}_\theta(\cdot, r, t), (\mathbf{v}_t, 0, 1))$
- 4 Set $\bar{\mathbf{u}}_{\text{tgt}} \leftarrow \mathbf{v}_t - (t - r) \frac{d}{dt}\bar{\mathbf{u}}_\theta$
- 5 Minimize $\|\bar{\mathbf{u}}_\theta(\mathbf{z}_t, r, t) - \text{sg}(\bar{\mathbf{u}}_{\text{tgt}})\|_2^2$ (optionally with adaptive weights)

Key innovations:

- **LogitNormal sampling**: Ensures proper time interval distribution
- **JVP computation**: Efficient higher-order derivatives
- **Stop-gradient**: Prevents training instability

Sampling (1 step or few steps)

Replace the time integral by the learned average velocity:

$$\mathbf{z}_r = \mathbf{z}_t - (t - r) \bar{\mathbf{u}}_\theta(\mathbf{z}_t, r, t). \quad (32)$$

1-NFE sampling: take $(r, t) = (0, 1)$, $\mathbf{z}_1 = \epsilon$,

$$\boxed{\mathbf{x} \equiv \mathbf{z}_0 = \epsilon - \bar{\mathbf{u}}_\theta(\epsilon, 0, 1)} . \quad (33)$$

Mathematical significance: This single-step generation maintains the same theoretical guarantees as multi-step Flow Matching.

Practical advantages:

- **Speed:** Single function evaluation
- **Stability:** No accumulation of integration errors
- **Flexibility:** Can still use few-step variants by chaining intervals

CFG inside the field (still 1-NFE)

Define a CFG velocity field

$$\mathbf{v}_{\text{cfg}}(\mathbf{z}_t, t \mid c) = \omega \mathbf{v}(\mathbf{z}_t, t \mid c) + (1 - \omega) \mathbf{v}(\mathbf{z}_t, t). \quad (34)$$

Key insight: CFG can be applied directly to the average velocity field:

$$\bar{\mathbf{u}}_{\text{cfg}}(\mathbf{z}_t, r, t \mid c) = \omega \bar{\mathbf{u}}(\mathbf{z}_t, r, t \mid c) + (1 - \omega) \bar{\mathbf{u}}(\mathbf{z}_t, r, t). \quad (35)$$

Mathematical property: This maintains the 1-NFE property while enabling conditional generation.

Practical significance: MeanFlow can achieve both single-step generation and strong conditioning, combining the best of both worlds.

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models
- 5 Practical Implementation
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions**
- 9 References

Key Takeaways: Flow Matching

Mathematical foundations:

- Flow Matching learns velocity fields that transport probability distributions via the continuity equation
- Conditional paths provide analytic supervision without score estimation
- The method connects to optimal transport theory and diffusion models

Computational advantages:

- No score estimation during training
- Deterministic sampling with flexible step sizes
- Stable gradients and efficient training

Practical benefits:

- Fewer function evaluations than diffusion models
- Better sample quality in many domains
- Natural extension to conditional generation

MeanFlow extension: Enables single-step generation while maintaining theoretical guarantees.

Future Directions and Open Problems

Theoretical developments:

- **Path optimization:** Finding optimal conditional paths for specific data distributions
- **Convergence analysis:** Understanding when Flow Matching achieves optimal transport
- **Error bounds:** Quantifying approximation errors in finite-sample settings

Practical improvements:

- **Architecture design:** Specialized networks for different data modalities
- **Training strategies:** Adaptive loss weighting and curriculum learning
- **Sampling efficiency:** Better ODE solvers and step size selection

Applications:

- **Scientific computing:** Molecular dynamics, fluid simulation
- **Creative AI:** Music, video, and 3D content generation
- **Data augmentation:** Synthetic data for training other models

Outline

- 1 Mathematical Foundations of Flow Matching
- 2 Conditional Paths and Velocity Targets
- 3 Flow Matching Training and Inference
- 4 Connection to Diffusion Models
- 5 Practical Implementation
- 6 Advanced Topics and Extensions
- 7 MeanFlow: Single-Step Generation
- 8 Summary and Future Directions
- 9 References

Key References

Core Flow Matching papers:

- Lipman et al., *Flow Matching: A unifying perspective of score-based training and generative modeling* (2023)
- Liu et al., *Flow Matching for Generative Modeling* (2023)
- Albergo et al., *Building Normalizing Flows with Stochastic Interpolants* (2023)

MeanFlow and extensions:

- Song et al., *MeanFlow: A Flow Matching Method for One-Step Generation* (2024)
- Song et al., *Rectified Flow: A Marginal Preserving Approach to Optimal Transport* (2022)

Related work:

- Song et al., *Score-based generative modeling through stochastic differential equations* (2021)
- Ho et al., *Denoising Diffusion Probabilistic Models* (2020)
- Chen et al., *Neural Ordinary Differential Equations* (2018)

Implementation resources:

- Flow Matching Guide and Code (GitHub)
- Open source libraries: diffusers, torchdyn

Welcome inputs

Any comments?

Welcome your inputs!